

Computación y Estructuras Discretas III

Andrés A. Aristizábal P. aaaristizabal@icesi.edu.co
Angela Villota - apvillota@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes



9 de febrero de 2026

1 Regular languages and Automata theory

- Deterministic Automata
- Exercises
- Non-deterministic Automata
- Exercises

1 Regular languages and Automata theory

- Deterministic Automata
- Exercises
- Non-deterministic Automata
- Exercises

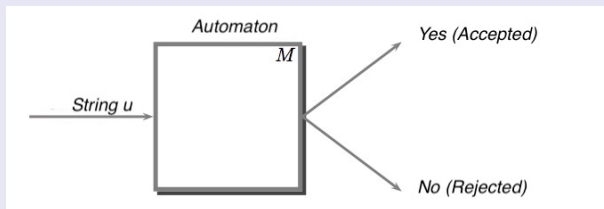
What is a finite automaton?

What is a finite automaton?

A finite automaton is an abstract machines that process input **strings**, which are either accepted or rejected.

What is a finite automaton?

A finite automaton is an abstract machines that process input **strings**, which are either accepted or rejected.



How does a finite automaton work?

How does a finite automaton work?

The automaton operates by reading the symbols written on a semi-infinite tape divided into cells, onto which an input string u is written, one symbol per cell.

How does a finite automaton work?

The automaton operates by reading the symbols written on a semi-infinite tape divided into cells, onto which an input string u is written, one symbol per cell.

What are the components of a finite automaton?

How does a finite automaton work?

The automaton operates by reading the symbols written on a semi-infinite tape divided into cells, onto which an input string u is written, one symbol per cell.

What are the components of a finite automaton?

The automaton has a control unit or reading head, which has a finite number of internal configurations, called states.

How does a finite automaton work?

The automaton operates by reading the symbols written on a semi-infinite tape divided into cells, onto which an input string u is written, one symbol per cell.

What are the components of a finite automaton?

The automaton has a control unit or reading head, which has a finite number of internal configurations, called states. There are some states of particular importance: these are the initial state and the final or accepting states.

What is the formal definition of a finite automaton?

What is the formal definition of a finite automaton?

Definition

A finite automaton M is defined as a 5-tuple, such that $M = (\Sigma, Q, q_0, F, \delta)$.

What is the formal definition of a finite automaton?

Definition

A finite automaton M is defined as a 5-tuple, such that $M = (\Sigma, Q, q_0, F, \delta)$.

- 1 *An alphabet Σ , called the tape alphabet. All strings processed by M belong to Σ^* .*

What is the formal definition of a finite automaton?

Definition

A finite automaton M is defined as a 5-tuple, such that $M = (\Sigma, Q, q_0, F, \delta)$.

- 1 An alphabet Σ , called the tape alphabet. All strings processed by M belong to Σ^* .*
- 2 $Q = q_0, q_1, \dots, q_n$, the set of internal states of the automaton.*

What is the formal definition of a finite automaton?

Definition

A finite automaton M is defined as a 5-tuple, such that $M = (\Sigma, Q, q_0, F, \delta)$.

- ❶ *An alphabet Σ , called the tape alphabet. All strings processed by M belong to Σ^* .*
- ❷ *$Q = q_0, q_1, \dots, q_n$, the set of internal states of the automaton.*
- ❸ *$q_0 \in Q$, the initial state.*

What is the formal definition of a finite automaton?

Definition

A finite automaton M is defined as a 5-tuple, such that $M = (\Sigma, Q, q_0, F, \delta)$.

- 1 An alphabet Σ , called the tape alphabet. All strings processed by M belong to Σ^* .
- 2 $Q = q_0, q_1, \dots, q_n$, the set of internal states of the automaton.
- 3 $q_0 \in Q$, the initial state.
- 4 $F \subseteq Q$, the set of final or accepting states. $F \neq \emptyset$.

What is the formal definition of a finite automaton?

Definition

A finite automaton M is defined as a 5-tuple, such that $M = (\Sigma, Q, q_0, F, \delta)$.

- 1 An alphabet Σ , called the tape alphabet. All strings processed by M belong to Σ^* .
- 2 $Q = q_0, q_1, \dots, q_n$, the set of internal states of the automaton.
- 3 $q_0 \in Q$, the initial state.
- 4 $F \subseteq Q$, the set of final or accepting states. $F \neq \emptyset$.
- 5 The transition function of the automaton:

$$\begin{array}{ccc} \delta : Q \times \Sigma & \longrightarrow & Q \\ & (q, s) \longmapsto & \delta(q, s) \end{array}$$

How does a finite automaton start working?

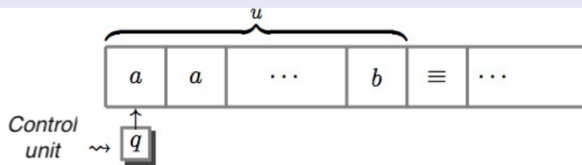
How does a finite automaton start working?

An input string u is placed on the tape in such a way that the first symbol of u occupies the first cell of the tape. The reading head is initially in the initial state q_0 .

Regular languages and Automata theory

How does a finite automaton start working?

An input string u is placed on the tape in such a way that the first symbol of u occupies the first cell of the tape. The reading head is initially in the initial state q_0 .



How does a finite automaton transition from state to state?

Regular languages and Automata theory

How does a finite automaton transition from state to state?

The transition function δ indicates the state to which the finite control moves, depending on the scanned symbol and its current state.

Regular languages and Automata theory

How does a finite automaton transition from state to state?

The transition function δ indicates the state to which the finite control moves, depending on the scanned symbol and its current state. So $\delta(q, s) = q'$, which indicates that when symbol s is encountered, the reading head moves from state q to q' , shifting to the right on the tape.

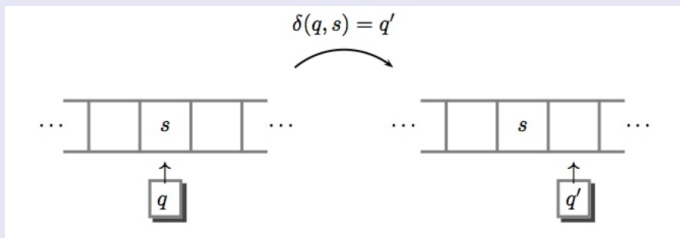
How does a finite automaton transition from state to state?

The transition function δ indicates the state to which the finite control moves, depending on the scanned symbol and its current state. So $\delta(q, s) = q'$, which indicates that when symbol s is encountered, the reading head moves from state q to q' , shifting to the right on the tape. This is one computational step.

Regular languages and Automata theory

How does a finite automaton transition from state to state?

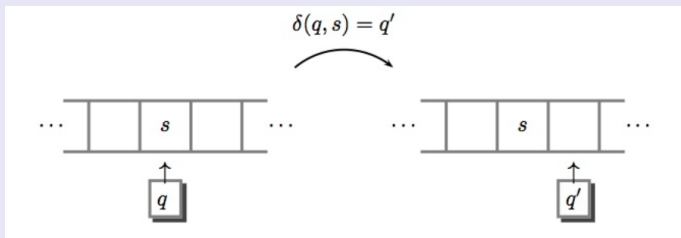
The transition function δ indicates the state to which the finite control moves, depending on the scanned symbol and its current state. So $\delta(q, s) = q'$, which indicates that when symbol s is encountered, the reading head moves from state q to q' , shifting to the right on the tape. This is one computational step.



Regular languages and Automata theory

How does a finite automaton transition from state to state?

The transition function δ indicates the state to which the finite control moves, depending on the scanned symbol and its current state. So $\delta(q, s) = q'$, which indicates that when symbol s is encountered, the reading head moves from state q to q' , shifting to the right on the tape. This is one computational step.



Since the transition function δ is defined for every state-symbol combination, an input string is processed entirely until the control unit encounters the empty string.

Example

Let's consider the automaton defined by the following 5 components:

$\Sigma = a, b$

$Q = q_0, q_1, q_2$

q_0 : *initial state.*

$F = q_0, q_2$, *accepting states.*

Transition function δ :

Example

Let's consider the automaton defined by the following 5 components:

$\Sigma = a, b$

$Q = q_0, q_1, q_2$

q_0 : initial state.

$F = q_0, q_2$, accepting states.

Transition function δ :

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_1

$$\delta(q_0, a) = q_0$$

$$\delta(q_0, b) = q_1$$

$$\delta(q_1, a) = q_1$$

$$\delta(q_1, b) = q_2$$

$$\delta(q_2, a) = q_1$$

$$\delta(q_2, b) = q_1.$$

Example

We illustrate the processing of two input strings.

Example

We illustrate the processing of two input strings.

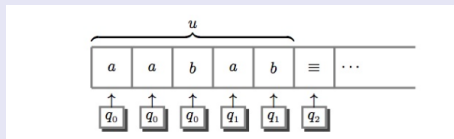
① $u = aabab$

Regular languages and Automata theory

Example

We illustrate the processing of two input strings.

① $u = aabab$

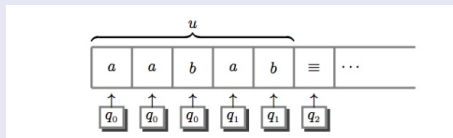


Regular languages and Automata theory

Example

We illustrate the processing of two input strings.

① $u = aabab$



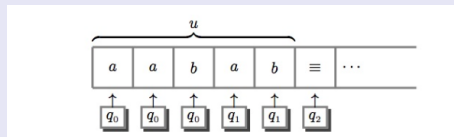
Since q_2 is an accepting state, the input string u is accepted.

Regular languages and Automata theory

Example

We illustrate the processing of two input strings.

① $u = aabab$



Since q_2 is an accepting state, the input string u is accepted.

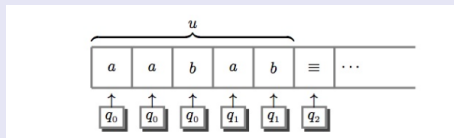
② $v = aababa$

Regular languages and Automata theory

Example

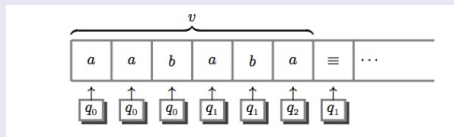
We illustrate the processing of two input strings.

❶ $u = aabab$



Since q_2 is an accepting state, the input string u is accepted.

❷ $v = aababa$

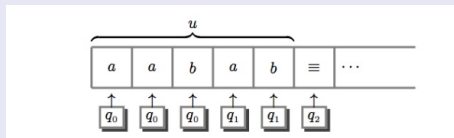


Regular languages and Automata theory

Example

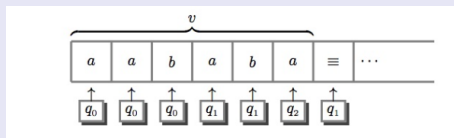
We illustrate the processing of two input strings.

❶ $u = aabab$



Since q_2 is an accepting state, the input string u is accepted.

❷ $v = aababa$



Since q_1 is not an accepting state, the string v is rejected.

How do we call these previously defined automata?

How do we call these previously defined automata?

They are called Deterministic Finite Automata (DFA) because for each state q and for each symbol $a \in \Sigma$, the transition function $\delta(q, a)$ is always defined.

How do we call these previously defined automata?

They are called Deterministic Finite Automata (DFA) because for each state q and for each symbol $a \in \Sigma$, the transition function $\delta(q, a)$ is always defined.

How do we call the language accepted by a finite automaton?

How do we call these previously defined automata?

They are called Deterministic Finite Automata (DFA) because for each state q and for each symbol $a \in \Sigma$, the transition function $\delta(q, a)$ is always defined.

How do we call the language accepted by a finite automaton?

Let M be an automaton, the language accepted by this automaton is denoted as $L(M)$, and it is defined as follows:

How do we call these previously defined automata?

They are called Deterministic Finite Automata (DFA) because for each state q and for each symbol $a \in \Sigma$, the transition function $\delta(q, a)$ is always defined.

How do we call the language accepted by a finite automaton?

Let M be an automaton, the language accepted by this automaton is denoted as $L(M)$, and it is defined as follows:

$$L(M) := \{u \in \Sigma^* \mid M \text{ terminates processing the input string } u \text{ in a state } q \in F\}.$$

What is a transition diagram of an automaton?

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

How is the transition diagram of an automaton obtained?

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

How is the transition diagram of an automaton obtained?

- *The vertices or nodes are the automaton's states*

Regular languages and Automata theory

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

How is the transition diagram of an automaton obtained?

- *The vertices or nodes are the automaton's states*

- *State q is represented by:* 

Regular languages and Automata theory

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

How is the transition diagram of an automaton obtained?

- The vertices or nodes are the automaton's states

- State q is represented by: 

- The initial state q_0 is represented by: 

Regular languages and Automata theory

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

How is the transition diagram of an automaton obtained?

- The vertices or nodes are the automaton's states

- State q is represented by: 

- The initial state q_0 is represented by: 

- The final state q is represented by: 

Regular languages and Automata theory

What is a transition diagram of an automaton?

It is a directed and labeled graph that represents the finite automaton.

How is the transition diagram of an automaton obtained?

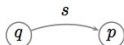
- The vertices or nodes are the automaton's states

- State q is represented by: 

- The initial state q_0 is represented by: 

- The final state q is represented by: 

- Transition $\delta = (q, s) = p$ is represented as:



Example

$\Sigma = \{a, b\}$.

$Q = \{q_0, q_1, q_2\}$.

q_0 : initial state

$F = \{q_0, q_2\}$, accepting states

Transition function δ :

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_1

$\delta(q_0, a) = q_0$

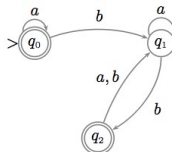
$\delta(q_0, b) = q_1$

$\delta(q_1, a) = q_1$

$\delta(q_1, b) = q_2$

$\delta(q_2, a) = q_1$

$\delta(q_2, b) = q_1$



1 Regular languages and Automata theory

- Deterministic Automata
- Exercises
- Non-deterministic Automata
- Exercises

1 Regular languages and Automata theory

- Deterministic Automata
- Exercises
- Non-deterministic Automata
- Exercises

What is a non-deterministic finite automaton?

Regular languages and Automata theory

What is a non-deterministic finite automaton?

The non-deterministic finite automata resemble deterministic finite automata, except that for each state $q \in Q$ and each $a \in \Sigma$, the transition $\delta(q, a)$ can consist of more than one state or may not be defined.

Regular languages and Automata theory

What is a non-deterministic finite automaton?

The non-deterministic finite automata resemble deterministic finite automata, except that for each state $q \in Q$ and each $a \in \Sigma$, the transition $\delta(q, a)$ can consist of more than one state or may not be defined.

How would we formally define a non-determinist finite automaton?

Regular languages and Automata theory

What is a non-deterministic finite automaton?

The non-deterministic finite automata resemble deterministic finite automata, except that for each state $q \in Q$ and each $a \in \Sigma$, the transition $\delta(q, a)$ can consist of more than one state or may not be defined.

How would we formally define a non-determinist finite automaton?

An NFA is defined by $M = (\Sigma, Q, q_0, F, \Delta)$ where:

Regular languages and Automata theory

What is a non-deterministic finite automaton?

The non-deterministic finite automata resemble deterministic finite automata, except that for each state $q \in Q$ and each $a \in \Sigma$, the transition $\delta(q, a)$ can consist of more than one state or may not be defined.

How would we formally define a non-determinist finite automaton?

An NFA is defined by $M = (\Sigma, Q, q_0, F, \Delta)$ where:

- 1 Σ is the tape alphabet.
- 2 Q is the (finite) set of internal states.
- 3 $q_0 \in Q$ is the initial state.
- 4 $\emptyset \neq F \subseteq Q$ is the set of final or accepting states.
- 5 Let $\mathcal{P}(Q)$ be the set of subsets of Q ,

$$\begin{aligned} \Delta : Q \times \Sigma &\longrightarrow \mathcal{P}(Q) \\ (q, s) &\mapsto \Delta(q, s) = \{q_{i1}, q_{i2}, \dots, q_{ik}\} \end{aligned}$$

What is the meaning $\Delta(q, s) = \{q_{i1}, q_{i2}, \dots, q_{ik}\}$?

What is the meaning $\Delta(q, s) = \{q_{i1}, q_{i2}, \dots, q_{ik}\}$?

Being in state q , upon encountering symbol s , it can randomly transition to any of the states $q_{i1}, q_{i2}, \dots, q_{ik}$ and moves to the right.

What is the meaning $\Delta(q, s) = \{q_{i1}, q_{i2}, \dots, q_{ik}\}$?

Being in state q , upon encountering symbol s , it can randomly transition to any of the states $q_{i1}, q_{i2}, \dots, q_{ik}$ and moves to the right.

What if $\Delta(q, s) = \emptyset$?

Regular languages and Automata theory

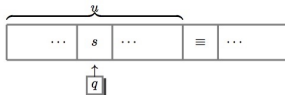
What is the meaning $\Delta(q, s) = \{q_{i1}, q_{i2}, \dots, q_{ik}\}$?

Being in state q , upon encountering symbol s , it can randomly transition to any of the states $q_{i1}, q_{i2}, \dots, q_{ik}$ and moves to the right.

What if $\Delta(q, s) = \emptyset$?

This means that if, during the processing of an input string u , M enters state q while reading symbol s on the tape, the computation is aborted.

Aborted computation

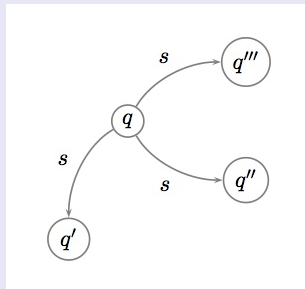


How do we represent an NFA by means of a transition diagram?

Regular languages and Automata theory

How do we represent an NFA by means of a transition diagram?

It is represented similarly to a DFA, the difference lies in the fact that from the same node (state), it may happen that two or more arcs with the same label emerge.



How does an FNA process an input string?

How does an FNA process an input string?

An NFA can process an input string $u \in \Sigma^*$ in different ways. Seen in the transition diagram, this means that there can be various paths from the initial state, labeled with the symbols of u .

What is the notion of acceptance for nondeterministic finite automata?

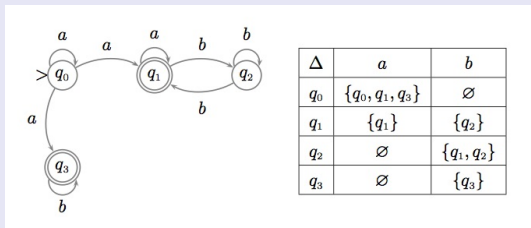
What is the notion of acceptance for nondeterministic finite automata?

$$\begin{aligned} L(M) &= \text{language accepted or recognized by } M \\ &= \{u \in \Sigma \mid \text{there exists at least one complete computation} \\ &\quad \text{of } u \text{ that ends in a state } q \in F\} \end{aligned}$$

For a string u to be accepted, there must exist some computation in which u is processed completely and ends with M in an accepting state.

Example

Let M be the following NFA:



For the string $u = abb$, there are computations that lead to rejection, aborted computations, and computations that end in accepting states.

Example

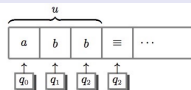
According to the definition of accepted language, $u \in L(M)$, we have:

Regular languages and Automata theory

Example

According to the definition of accepted language, $u \in L(M)$, we have:

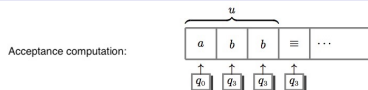
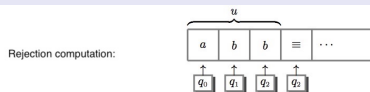
Rejection computation:



Regular languages and Automata theory

Example

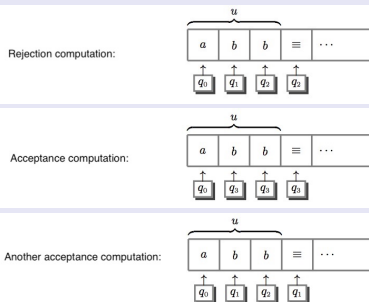
According to the definition of accepted language, $u \in L(M)$, we have:



Regular languages and Automata theory

Example

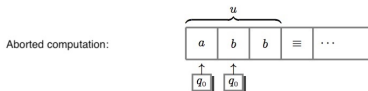
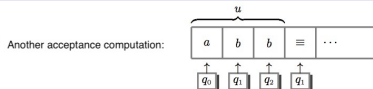
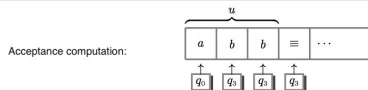
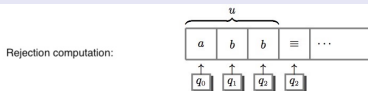
According to the definition of accepted language, $u \in L(M)$, we have:



Regular languages and Automata theory

Example

According to the definition of accepted language, $u \in L(M)$, we have:



Are DFAs and NFAs computationally equivalent?

Are DFAs and NFAs computationally equivalent?

The DFAs and NFAs are computationally equivalent.

Are DFAs and NFAs computationally equivalent?

The DFAs and NFAs are computationally equivalent. It can be seen that a DFA $M = (\Sigma, Q, q_0, F, \delta)$ can be considered as an NFA $M' = (\Sigma, Q, q_0, F, \Delta)$ by defining $\Delta(q, a) = \delta(q, a)$ for each $q \in Q$ and each $a \in \Sigma$

Are DFAs and NFAs computationally equivalent?

The DFAs and NFAs are computationally equivalent. It can be seen that a DFA $M = (\Sigma, Q, q_0, F, \delta)$ can be considered as an NFA $M' = (\Sigma, Q, q_0, F, \Delta)$ by defining $\Delta(q, a) = \delta(q, a)$ for each $q \in Q$ and each $a \in \Sigma$

For the converse of this implication, we have the following theorem:

Are DFAs and NFAs computationally equivalent?

The DFAs and NFAs are computationally equivalent. It can be seen that a DFA $M = (\Sigma, Q, q_0, F, \delta)$ can be considered as an NFA $M' = (\Sigma, Q, q_0, F, \Delta)$ by defining $\Delta(q, a) = \delta(q, a)$ for each $q \in Q$ and each $a \in \Sigma$

For the converse of this implication, we have the following theorem:

Teorema

Given an NFA $M = (\Sigma, Q, q_0, F, \Delta)$, we can construct an equivalent DFA M' to M , meaning that $L(M) = L(M')$

What is a nondeterministic finite automaton with λ transitions?

What is a nondeterministic finite automaton with λ transitions?

It is a nondeterministic automaton $M = (\Sigma, Q, q_0, F, \Delta)$ where the transition function is defined as $\Delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$.

What is a nondeterministic finite automaton with λ transitions?

It is a nondeterministic automaton $M = (\Sigma, Q, q_0, F, \Delta)$ where the transition function is defined as $\Delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$.

What is a λ transition?

What is a nondeterministic finite automaton with λ transitions?

It is a nondeterministic automaton $M = (\Sigma, Q, q_0, F, \Delta)$ where the transition function is defined as $\Delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$.

What is a λ transition?

The λ transition, $\Delta(q, \lambda) = \{p_{i_1}, \dots, p_{i_n}\}$, also called null or spontaneous transition, allows transitioning between states without processing any symbol read on the tape.

What is a nondeterministic finite automaton with λ transitions?

It is a nondeterministic automaton $M = (\Sigma, Q, q_0, F, \Delta)$ where the transition function is defined as $\Delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$.

What is a λ transition?

The λ transition, $\Delta(q, \lambda) = \{p_{i_1}, \dots, p_{i_n}\}$, also called null or spontaneous transition, allows transitioning between states without processing any symbol read on the tape.

How do we represent lambda transitions in a transition diagram?

What is a nondeterministic finite automaton with λ transitions?

It is a nondeterministic automaton $M = (\Sigma, Q, q_0, F, \Delta)$ where the transition function is defined as $\Delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$.

What is a λ transition?

The λ transition, $\Delta(q, \lambda) = \{p_{i_1}, \dots, p_{i_n}\}$, also called null or spontaneous transition, allows transitioning between states without processing any symbol read on the tape.

How do we represent lambda transitions in a transition diagram?

In a transition diagram, these transitions result in arcs labeled with λ .

When is an input w accepted by an NFA- λ ?

When is an input w accepted by an NFA- λ ?

A string w is accepted by an NFA- λ if there exists at least one path from the initial state q_0 to an accepting state, whose labels consist exactly of the symbols of w , interspersed with zero or more λ s.

When is an input w accepted by an NFA- λ ?

A string w is accepted by an NFA- λ if there exists at least one path from the initial state q_0 to an accepting state, whose labels consist exactly of the symbols of w , interspersed with zero or more λ s.

Example

Let M be the following NFA- λ :

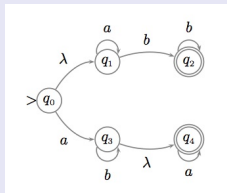
Regular languages and Automata theory

When is an input w accepted by an NFA- λ ?

A string w is accepted by an NFA- λ if there exists at least one path from the initial state q_0 to an accepting state, whose labels consist exactly of the symbols of w , interspersed with zero or more λ s.

Example

Let M be the following NFA- λ :

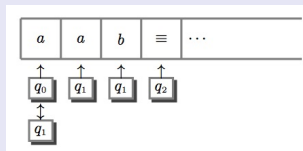


Example

The acceptance computation for $u = aab$ is:

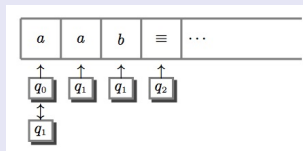
Example

The acceptance computation for $u = aab$ is:



Example

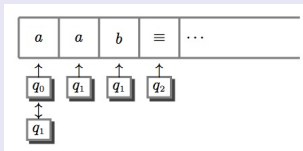
The acceptance computation for $u = aab$ is:



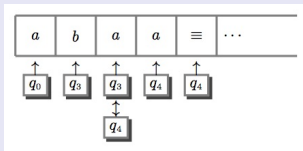
The acceptance computation for $v = abaa$ is:

Example

The acceptance computation for $u = aab$ is:



The acceptance computation for $v = abaa$ is:



What advantages does an NFA- λ have?

Regular languages and Automata theory

What advantages does an NFA- λ have?

It allows even more freedom in the design of automata, especially when there are numerous concatenations.

Regular languages and Automata theory

What advantages does an NFA- λ have?

It allows even more freedom in the design of automata, especially when there are numerous concatenations.

Example

*Let M be the following DFA that accepts the language $a^*b^*c^*$ over $\Sigma = \{a, b, c\}$:*

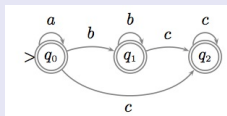
Regular languages and Automata theory

What advantages does an NFA- λ have?

It allows even more freedom in the design of automata, especially when there are numerous concatenations.

Example

Let M be the following DFA that accepts the language $a^*b^*c^*$ over $\Sigma = \{a, b, c\}$:



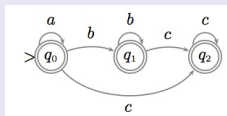
Regular languages and Automata theory

What advantages does an NFA- λ have?

It allows even more freedom in the design of automata, especially when there are numerous concatenations.

Example

Let M be the following DFA that accepts the language $a^*b^*c^*$ over $\Sigma = \{a, b, c\}$:



M' is an NFA- λ that accepts L :

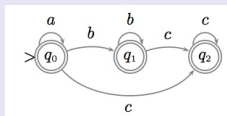
Regular languages and Automata theory

What advantages does an NFA- λ have?

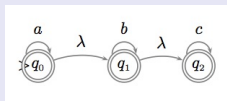
It allows even more freedom in the design of automata, especially when there are numerous concatenations.

Example

Let M be the following DFA that accepts the language $a^*b^*c^*$ over $\Sigma = \{a, b, c\}$:



M' is an NFA- λ that accepts L :

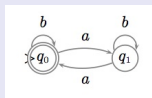


Example

Let M_1 be the following DFA that accepts the language of all strings with an even number of a 's over $\Sigma = \{a, b\}$:

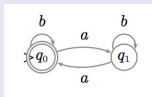
Example

Let M_1 be the following DFA that accepts the language of all strings with an even number of a 's over $\Sigma = \{a, b\}$:



Example

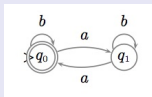
Let M_1 be the following DFA that accepts the language of all strings with an even number of a 's over $\Sigma = \{a, b\}$:



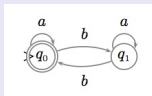
And let M_2 be the following DFA that accepts the language of all strings with an even number of b 's over $\Sigma = \{a, b\}$:

Example

Let M_1 be the following DFA that accepts the language of all strings with an even number of a 's over $\Sigma = \{a, b\}$:



And let M_2 be the following DFA that accepts the language of all strings with an even number of b 's over $\Sigma = \{a, b\}$:

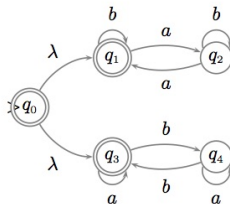


Example

M_3 is the following NFA- λ that accepts the language of all strings with an even number of a 's or an even number of b 's over $\Sigma = \{a, b\}$:

Example

M_3 is the following NFA- λ that accepts the language of all strings with an even number of a 's or an even number of b 's over $\Sigma = \{a, b\}$:



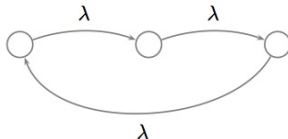
Can infinite loops occur in an NFA- λ ?

Can infinite loops occur in an NFA- λ ?

Unlike DFAs and NFAs, in NFAs- λ , there can be computations that never terminate. This can happen if the automaton enters a state from which there are multiple chained λ transitions that return to the same state.

Can infinite loops occur in an NFA- λ ?

Unlike DFAs and NFAs, in NFAs- λ , there can be computations that never terminate. This can happen if the automaton enters a state from which there are multiple chained λ transitions that return to the same state.



Are NFAs and NFAs- λ computationally equivalent?

Are NFAs and NFAs- λ computationally equivalent?

NFA- λ are computationally equivalent to NFAs. In other words, λ transitions can be eliminated by adding transitions that simulate them, without altering the accepted language.

Are NFAs and NFAs- λ computationally equivalent?

NFA- λ are computationally equivalent to NFAs. In other words, λ transitions can be eliminated by adding transitions that simulate them, without altering the accepted language.

An NFA $M = (\Sigma, Q, q_0, F, \Delta)$ can be considered as an NFA- λ where there are simply zero λ transitions.

Are NFAs and NFAs- λ computationally equivalent?

NFA- λ are computationally equivalent to NFAs. In other words, λ transitions can be eliminated by adding transitions that simulate them, without altering the accepted language.

An NFA $M = (\Sigma, Q, q_0, F, \Delta)$ can be considered as an NFA- λ where there are simply zero λ transitions.

For the converse of this implication, we have the following theorem:

Are NFAs and NFAs- λ computationally equivalent?

NFA- λ are computationally equivalent to NFAs. In other words, λ transitions can be eliminated by adding transitions that simulate them, without altering the accepted language.

An NFA $M = (\Sigma, Q, q_0, F, \Delta)$ can be considered as an NFA- λ where there are simply zero λ transitions.

For the converse of this implication, we have the following theorem:

Teorema

Given an NFA- λ $M = (\Sigma, Q, q_0, F, \Delta)$, we can construct an equivalent NFA M' to M , meaning that $L(M) = L(M')$.

1 Regular languages and Automata theory

- Deterministic Automata
- Exercises
- Non-deterministic Automata
- Exercises